Ion Traps with QuTiP

Simulating trapped-ion quantum computers from first principles

Spencer Churchill | Staff Software Engineer, IonQ

QuTiP Workshop 2026 · RIKEN, Japan

Talk outline

1. Live demo - Bell state in 10 lines
2. The physics - traps, crystals, modes, interactions
3. Building the simulator layer by layer (with bug stories)
4. What this means for QuTiP

Bell state in 10 lines

```
from tiqs import PaulTrap, SimulationConfig, SimulationRunner, get_species
from tiqs.analysis.fidelity import bell_state_fidelity

TWO_PI = 2 * 3.141592653589793
species = get_species("Ca40")
trap = PaulTrap(v_rf=300, omega_rf=TWO_PI * 30e6, r0=0.5e-3, omega_axial=TWO_PI * 1e6, species=species)
config = SimulationConfig(species=species, trap=trap, n_ions=2, n_modes=1, n_fock=15)
runner = SimulationRunner(config)
result = runner.run_ms_gate(ions=[0, 1])
print(bell_state_fidelity(result.states[-1].ptrace([0, 1])))
```

```
>>> 1.0000000149010342
```

Two Ca-40 ions. One perfect Bell state. 34 modules, 178 tests underneath.

What is a trapped-ion quantum computer?

Ions confined in an RF Paul trap. Qubits encoded in electronic states. Entanglement mediated by shared motional modes. Everything reduces to time-dependent Hamiltonians and Lindblad channels - which is exactly what QuTiP solves.

RF Paul trap	Confines ions in a linear crystal with oscillating electric fields
Laser beams	Drive qubit transitions and couple internal states to motion
Vibrational modes	Shared motional modes act as an entanglement bus
Decoherence	Heating, dephasing, photon scattering - modeled as Lindblad channels

The Paul trap

```
trap = PaulTrap(
    v_rf=300,                # 300 V RF amplitude
    omega_rf=TWO_PI*30e6,    # 30 MHz drive
    r0=0.5e-3,               # 0.5 mm electrode distance
    omega_axial=TWO_PI*1e6,  # 1 MHz axial frequency
    species=get_species("Ca40"),
)
trap.mathieu_q     # 0.163  (stability parameter)
trap.omega_radial  # 2π × 1.579 MHz
trap.is_stable()   # True
```

Mathieu stability

$$q = 2eV / (m \cdot \Omega^2 \cdot r_0^2)$$

Secular frequency

$$\omega_r = (\Omega/2) \cdot \sqrt{a + q^2/2}$$

QuTiP doesn't care about traps - but to build correct Hamiltonians, you need correct frequencies.

Coulomb crystals and normal modes

Two Ca-40 ions at 1 MHz axial frequency sit 5.6 µm apart. Small oscillations decompose into normal modes via the Hessian of the total potential.

```
modes = normal_modes(2, trap)
modes.axial_freqs[0] / trap.omega_axial  # 1.000000 (COM)
modes.axial_freqs[1] / trap.omega_axial  # 1.732051 (stretch)
np.sqrt(3)                               # 1.732051
```

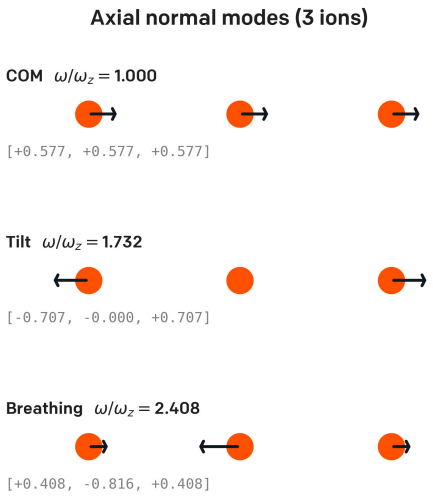
The stretch-to-COM frequency ratio is exactly $\sqrt{3}$ - reproduced to machine precision (15 digits, 1.1e-15 absolute error).

COM mode: both ions in phase. Stretch mode: ions out of phase.

Three-ion normal modes

```
modes3 = normal_modes(3, trap)
# Positions: [-4.793, 0.000, 4.793] μm
# Frequency ratios: [1.000, 1.732, 2.408]
```

Mode	Eigenvector	Physical picture
COM	[0.577, 0.577, 0.577]	All ions in phase
Tilt	[-0.707, 0.000, 0.707]	Ends oppose, center stationary
Breathing	[0.408, -0.816, 0.408]	Center opposes both ends



These eigenvectors are the participation matrix b_{i,m} that determines everything about the quantum gates.

The Lamb-Dicke parameter

The single most important number in trapped-ion physics:

$$\eta_{i,m} = k_{eff} \cdot b_{i,m} \cdot \sqrt{\hbar / 2m \cdot \omega_m}$$

```
k_eff = TWO_PI / 729e-9  # Ca-40 qubit laser
eta = lamb_dicke_parameters(modes, species, k_eff, "axial")
# [[-0.069, -0.052],
#  [-0.069,  0.052]]
```

COM mode: same sign - both ions couple equally.
Stretch mode: opposite signs - this is what makes the entangling gate work.

Bug story: the k_eff factor of 2

For hyperfine qubits (Yb-171), counter-propagating Raman beams mean $k_{eff} = 2\cdot(2\pi/\lambda)$, not $k_{eff} = 2\pi/\lambda$.

```
# Before (wrong):
k_eff = TWO_PI / raman_wavelength

# After (correct):
k_eff = 2 * TWO_PI / raman_wavelength
```

Wrong for weeks. The gate still worked - just at the wrong Rabi frequency. The error only showed up when matching published Lamb-Dicke parameters from the Monroe group.

Six ion species

Species	Qubit type	Qubit freq / λ	Cooling	Raman
Yb-171	Hyperfine	12.643 GHz	369.5 nm	355 nm
Ca-40	Optical	729 nm	397 nm	-
Ca-43	Hyperfine	3.226 GHz	397 nm	397 nm
Ba-137	Hyperfine	8.038 GHz	493 nm	515 nm
Be-9	Hyperfine	1.25 GHz	313 nm	313 nm
Sr-88	Optical	674 nm	422 nm	-

Every linewidth is from precision lifetime measurements. Yb-171 uses Olmschenk 2009 (8.12 ns -> 19.6 MHz), not the 23 MHz found in older references - a 15% difference that changes everything downstream.

Module architecture

```
src/tiqs/
    species/       IonSpecies, Transition      -> atomic data
    trap.py        PaulTrap                    -> Mathieu, secular freqs
    chain/         equilibrium, normal_modes   -> Coulomb crystal physics
    hilbert_space/ HilbertSpace, Operators     -> qutip.tensor, qeye
    interaction/   carrier, sidebands, full H  -> [Qobj, "coeff(t)"] lists
    gates/         MS, light-shift, CZ, Rx/y/z -> qutip.sesolve
    cooling/       Doppler, SBC, EIT           -> rate equations, mesolve
    noise/         8 error sources             -> Lindblad c_ops
    spam/          pumping, fluorescence       -> ptrace, ket2dm
    transport.py   shuttling, splitting        -> phenomenological models
    simulation/    SimulationRunner            -> sesolve / mesolve / mcsolve
    analysis/      fidelity, Wigner, errors    -> qutip.fidelity, wigner
```

Not a wrapper around QuTiP - a physics layer that feeds QuTiP.

Which QuTiP features get exercised

Module	QuTiP feature
hilbert_space/	tensor, qeye, mixed-dim spaces [2, 2, 15]
interaction/	Time-dependent H: [Qobj, "exp(1j*delta*t)"]
gates/	sesolve with string coefficients
noise/	Lindblad collapse operators, mesolve
simulation/	sesolve vs mesolve vs mcsolve dispatch
spam/	ptrace, density matrix projectors
analysis/	fidelity, wigner, thermal_dm

Exercises nearly the full surface area of QuTiP's core API.

The Hilbert space

Convention: [qubit_0, qubit_1, …, mode_0, mode_1, …]

```
hs = HilbertSpace(n_ions=2, n_modes=1, n_fock=15)
hs.dims       # [2, 2, 15]
hs.total_dim  # 60

ops = OperatorFactory(hs)
ops.sigma_x(0)     # Pauli X on ion 0, I on rest
ops.annihilate(0)  # a on mode 0, I on qubits

sf = StateFactory(hs)
sf.ground_state()            # |0, 0, n=0>
sf.thermal_state(n_bar=[5])  # thermal motional state
```

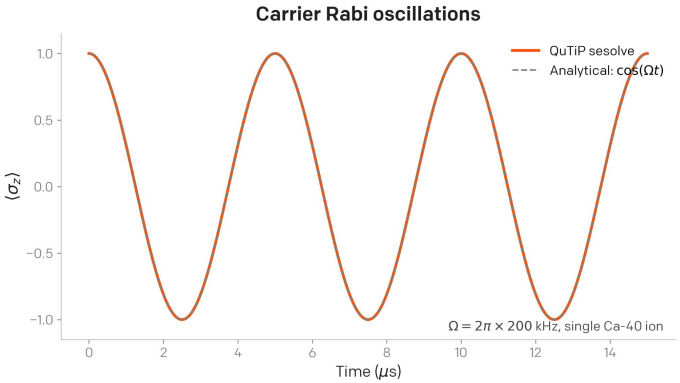
Discussion: would it make sense for QuTiP to have named subsystems? Something like system.qubit[0] instead of tracking indices manually.

Carrier Rabi oscillations

```
Omega = TWO_PI * 200e3  # 200 kHz Rabi frequency
H = carrier_hamiltonian(ops, ion=0, rabi_frequency=Omega)
psi0 = sf.ground_state()

tlist = np.linspace(0, 3 * TWO_PI / Omega, 500)
result = qutip.sesolve(H, psi0, tlist, e_ops=[ops.sigma_z(0)])
```

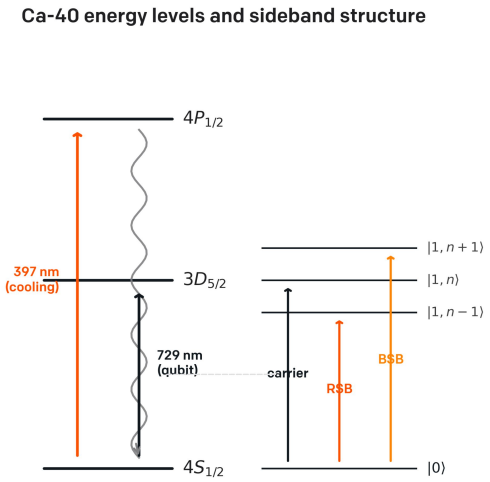
$H = (\Omega/2)\cdot\sigma_x$. Starting from $|0\rangle$, expectation oscillates as $\cos(\Omega\cdot t)$.

Analytical check: simulated curve matches $\cos(\Omega\cdot t)$ to 1% over 500 time points.



Laser-ion interaction: three resonances

```
H = full_interaction_hamiltonian(
    ops, ion=0, mode=0, rabi_frequency=Omega,
    eta=0.1, detuning=delta, mode_frequency=omega_m,
)
# Returns QuTiP list-format:
# [ H_carrier,
#   [H_rsb_up,   "exp(1j*(delta-omega_m)*t)"],
#   [H_rsb_down, "exp(-1j*(delta-omega_m)*t)"],
#   [H_bsb_up,   "exp(1j*(delta+omega_m)*t)"],
#   [H_bsb_down, "exp(-1j*(delta+omega_m)*t)"],
# ]
```



Carrier (δ=0) · Red sideband (δ=ωm) · Blue sideband (δ=−ωm)

QuTiP's string-coefficient time dependence handles the complex exponentials.

Second-order Lamb-Dicke corrections

```
H2 = full_interaction_hamiltonian(
    ops, ion=0, mode=0, Omega, eta, delta, omega_m,
    lamb_dicke_order=2,  # adds 3 more terms
)
```

Debye-Waller correction:

$$\Omega_eff(n) = \Omega \cdot (1 - \eta^2 \cdot (n + ½))$$

The second-order expansion of $\exp(i\eta(a + a\dagger))$ gives:
$(i\eta)^2/2! \cdot (a + a\dagger)^2 = -\eta^2/2 \cdot (a^2 + a\dagger^2 + 2n + 1)$

Bug story: the √2 and the zero-point energy

Bug 1: second-sideband coefficient was $\eta^2 \cdot \Omega/(2 \cdot \sqrt{2})$ instead of $\eta^2 \cdot \Omega/4$.
An extra √2 from confusing matrix elements.

Bug 2: Debye-Waller term was $-\eta^2 \cdot n$ instead of $-\eta^2 \cdot (n + ½)$.
The zero-point energy term 0.5·identity() was missing.

```
# Bug 1: η²·Ω / (2·√2)  ->  η²·Ω / 4
# Bug 2: -η²·n          ->  -η²·(n + ½)
```

Both are one-line fixes. Both took days to find. The second one only showed up when comparing to published thermal-state gate fidelity curves.

Three entangling gate protocols

Gate	Interaction	Drive	Key property
Mølmer-Sørensen	$\sigma_x \otimes \sigma_x$	Bichromatic	Insensitive to initial motion
Light-shift	$\sigma_z \otimes \sigma_z$	AC Stark	Insensitive to optical phase
Cirac-Zoller	Sequential RSB	Single-beam	Requires ground-state cooling

Different physics, same QuTiP interface: construct a list-format H, pass to sesolve or mesolve.

The Mølmer-Sørensen gate

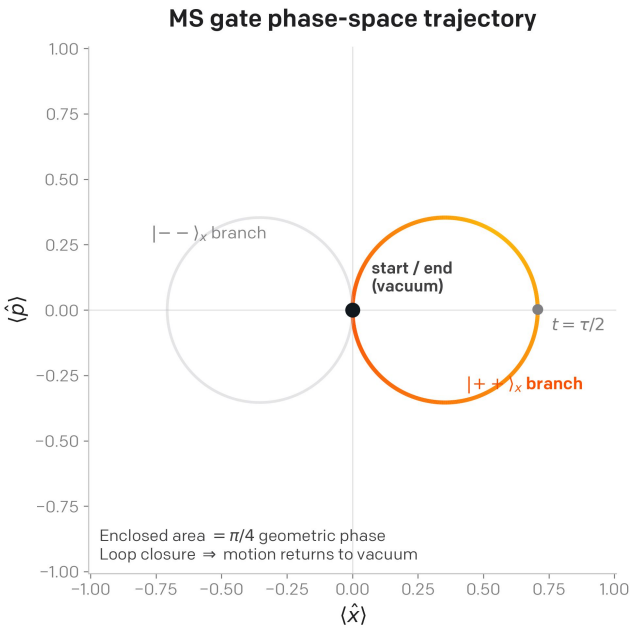
```
hs = HilbertSpace(n_ions=2, n_modes=1, n_fock=20)
ops = OperatorFactory(hs)
sf = StateFactory(hs)

eta = 0.05
delta = TWO_PI * 15e3  # 15 kHz detuning from sideband
Omega = delta / (4 * eta)  # 75 kHz - maximally entangling
tau = ms_gate_duration(delta)  # 66.7 us

H = ms_gate_hamiltonian(ops, [0, 1], 0, [eta, eta], Omega, delta)
result = qutip.sesolve(H, sf.ground_state(), np.linspace(0, tau, 500), options={"max_step": tau / 100})

bell_state_fidelity(result.states[-1].ptrace([0, 1]))
# 1.0000000290
```

```
Bell state fidelity: 1.0000000290
```

How the MS gate works

Spin-dependent force displaces motional mode in opposite directions for different σx eigenstates, tracing circles in phase space. Condition $\eta \cdot \Omega = \delta/4$ ensures the circle closes after one loop ($\tau = 2\pi/\delta$), accumulating geometric phase $\pi/4$.

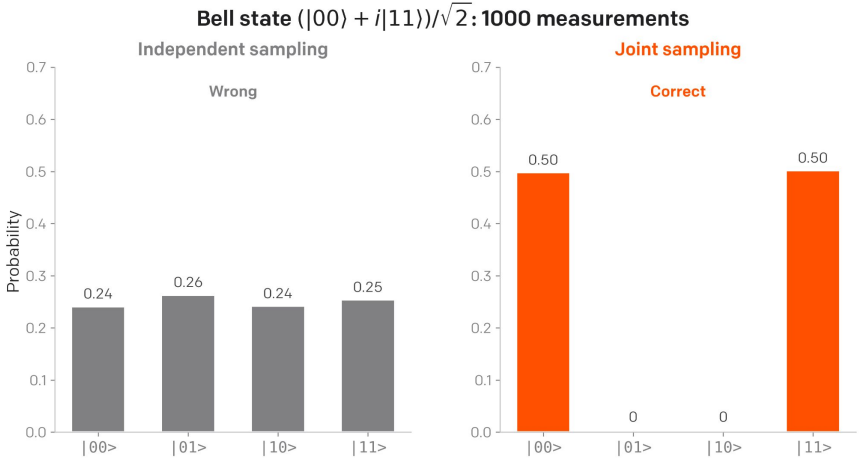
Result: $(|00\rangle + i|11\rangle) / \sqrt{2}$



The symmetric dip to 0.56 and recovery to 1.0 is the phase-space loop closure.

Bug story: measurement destroys entanglement

```
# WRONG: samples each ion independently
for ion in ions:
    p_bright = rho.ptrace(ion)[0, 0].real
    bit = 0 if rng.random() < p_bright else 1

# CORRECT: sample from the joint probability distribution
rho_ions = rho.ptrace(ions)
probs = [rho_ions[i, i].real for i in range(4)]
outcome_idx = rng.choice(4, p=probs)
```



For a Bell state, each ion has 50/50 probability. Independent sampling produces 01 and 10 - which never occur in the actual state.

This bug doesn't show up in fidelity calculations - only in measurement statistics. Our end-to-end test caught it: 200 samples must produce >195 correlated outcomes.

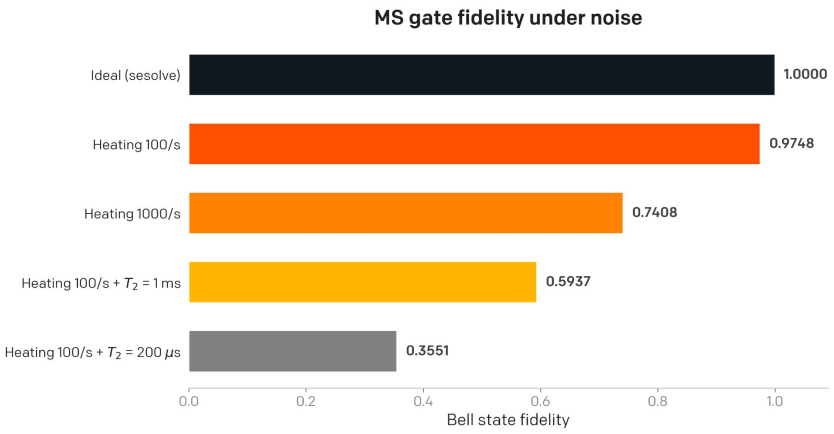
6 Lindblad channels + 2 coherent errors

Source	Type	Operator	Typical value
Motional heating	Lindblad	$\sqrt{(\dot{n})}\cdot a\dagger$	10-1000 q/s
Motional dephasing	Lindblad	$\sqrt{(\text{rate})}\cdot n$	trap freq fluct.
Qubit dephasing	Lindblad	$\sqrt{((1/T_2-1/2T_1)/2)}\cdot\sigma z$	T_2 ~ 0.1-10 ms
Spontaneous emission	Lindblad	$\sqrt{(1/T_1)}\cdot\sigma+$	T_1 ~ 1 s
Rayleigh scattering	Lindblad	$\sqrt{(\text{rate}/4)}\cdot\sigma z$	during gates
Raman scattering	Lindblad	$\sqrt{(\text{rate})}\cdot\sigma+$	fundamental
Intensity noise	Hamiltonian	$(\delta\Omega/2)\cdot\sigma x$	~0.01 frac.
Addressing crosstalk	Hamiltonian	$(\varepsilon\cdot\Omega/2)\cdot\sigma x$	$\varepsilon \sim 10^{-3}$

One line of physics, one line of QuTiP.

Noisy MS gate

```
config = SimulationConfig(
    species=get_species("Ca40"), trap=trap,
    n_ions=2, n_modes=1, n_fock=15,
    solver="mesolve",
    heating_rate=100,  t2_qubit=1e-3,
)
runner = SimulationRunner(config)
result = runner.run_ms_gate(ions=[0, 1])
```



Turn noise sources on one at a time and you get an error budget.

Cooling pipeline

From room temperature (~10^7 phonons) to near ground state:

```
# Stage 1: Doppler cooling
n_doppler = doppler_cooled_nbar(species, trap_frequency_hz=1e6)
# Ca-40 at 1 MHz: n_bar = 11.2

# Stage 2: Sideband cooling (simulated with QuTiP mesolve)
n_sbc = sideband_cooling_simulate(ops, ion=0, mode=0,
    n_bar_initial=5.0, eta=0.1, rabi_frequency=TWO_PI*100e3,
    optical_pumping_rate=TWO_PI*10e3, n_cycles=30)
# n_bar drops from 5.0 to < 0.5

# Stage 3: Analytical EIT cooling limit
n_eit = eit_cooling_nbar(TWO_PI*200e3, TWO_PI*1e6,
    carrier_suppression=0.01)
# n_bar ~ 0.001
```

State preparation and measurement

Preparation (optical pumping):

```
c_ops = [sqrt(pumping_rate) * ops.sigma_plus(ion)]
rho_prepared = qutip.mesolve(0*H, rho_initial, tlist, c_ops=c_ops)
```

Measurement (joint Born-rule sampling):

```
rho_ions = rho.ptrace(ions)
probs = [rho_ions[i, i].real for i in range(2**n)]
outcome = rng.choice(2**n, p=probs)
```

Mid-circuit measurement (Born-rule projection):

```
P_k = tensor(I, ..., |k><k|, ..., I)
rho_post = P_k * rho * P_k / Tr(P_k * rho)
```

All ptrace, ket2dm, tensor - core QuTiP operations.

Composite pulse sequences

```
sk1 = sk1_composite_gate(ops, ion=0, theta=np.pi)
# Returns 3 sequential pulses:
#   R_0(π), R_φ1(2π), R_{-φ1}(2π)
# where φ1 = arccos(-θ / (4π))
```

Each pulse is a (Hamiltonian, duration) pair. With a 5% Rabi frequency error:

Gate	Infidelity (5% amp error)
Bare Rx(π)	4.1×10^{-3}
SK1(π)	9.5×10^{-5} (~43× better)

SK1 cancels amplitude errors to first order. BB1 cancels to second order.

Transport (QCCD)

```
delta_n = shuttle_motional_excitation(
    distance=200e-6, duration=50e-6,
    trap_frequency=TWO_PI*1e6,
)
# Returns estimated added quanta (exponential adiabaticity model)

rho_after = apply_shuttling_noise(
    rho, ops, mode=0, added_quanta=delta_n,
)
# Applies thermal noise via Lindblad channel: γ·t = ln(Δn + 1)
```

Phenomenological noise model - not a time-dependent potential simulation. Captures the key effect: transport adds motional excitation that degrades subsequent gates.

What works beautifully in QuTiP

String-based time dependence

The single most important QuTiP feature for AMO physics. The MS gate Hamiltonian is just [[η*Ω*a†*σx, "exp(1j*δ*t)"], ...] and QuTiP handles compilation, ODE integration, everything.

The solver triad

sesolve (pure state), mesolve (Lindblad), mcsolve (trajectories) with compatible interfaces. Switch by changing one config parameter.

ptrace everywhere

Tracing out motional modes for spin fidelity, tracing out other ions for single-qubit measurement. The workhorse of trapped-ion simulation.

What could be better

1. Named subsystems in tensor products

Eliminate indexing bugs - system.qubit[0] vs tracking indices manually

2. Joint measurement sampling

qutip.measurement.sample(rho, subsystems) - avoids correlation bug

3. Hamiltonian builder pattern

HamiltonianBuilder().add(H, coeff) with validation, not list concatenation

4. Parametric time-dependent Hamiltonians

Parameterized coefficients for efficient parameter sweeps

5. ptrace performance on deep products

For 5+ ions with modes (dim ~10^5), ptrace starts to dominate runtime

178 tests, 99% coverage

13 analytical exactness checks against closed-form results:

Analytical check	Tolerance
2-ion spacing vs dimensionless solution	0.1%
Stretch/COM frequency ratio = $\sqrt{3}$	Machine precision
Lamb-Dicke parameter vs hand calculation	10^{-6} relative
Carrier $\sigma z = \cos(\Omega \cdot t)$	1% over 500 pts
RSB Rabi freq = $\eta \cdot \Omega \cdot \sqrt{n}$ for n=1,2,3,5	>98% fidelity
MS gate Bell fidelity	> 0.999
Dephasing decay = $\exp(-t/T_2)$	5%
Heating $\langle n \rangle(t) = \exp(\gamma t) - 1$	5%
Bell measurement: >195/200 correlated	Statistical

Not unit testing - physics validation.

In the QuTiP ecosystem

BSD-3 licensed · pip install · depends only on QuTiP
32 modules, 2,721 lines, 178 tests, 99% coverage
6 ion species · 6 decoherence channels · 3 entangling gates · 3 cooling stages

Possible roles:

- A qutip-qip trapped-ion processor backend
- A reference implementation for trapped-ion physics with QuTiP
- Benchmark test cases for solver development
- A community-maintained ion species database

Built for my work at IonQ. Sharing it because the QuTiP community should own the canonical trapped-ion simulation layer.

Three questions for this group

For the solver developers:

What's the most efficient way to handle multi-term time-dependent Hamiltonians with 5-10 oscillating components? Is the string coefficient approach the right one, or should I be using something else in QuTiP 5? Would Krylov solvers help?

For the qutip-qip maintainers:

Would it make sense to add a TIQS-based trapped-ion processor backend? The Processor class abstracts over pulse-level physics - TIQS provides exactly that layer.

For everyone:

Is there interest in a "named subsystem" feature for composite Hilbert spaces in core QuTiP? Or should that stay in application packages like TIQS?

```
>>> bell_state_fidelity(result.states[-1].ptrace([0, 1]))
1.0000000290
```

Mathieu stability -> Coulomb equilibria -> Hessian eigendecomposition ->
Lamb-Dicke parameters -> interaction-picture Hamiltonians ->
phase-space loop closure -> six Lindblad channels -> sesolve

The physics is real. The simulator is open source. Every formula is validated.

Thank you. And thank you to the QuTiP team for building a framework where this kind of thing is possible.

github.com/splch/tiqs